Muhammed kallo 220201916

Sudenaz Kılıç 220201054

Gülbahar Yalçındağ 220201001

Nisa Sert 220201029

MLRP-KG: Mine Landslide Risk Prediction Based on Knowledge Graph

1. We propose a new mine landslide risk prediction model using knowledge graph

In this study, a Knowledge Graph (KG)-based model is proposed to predict landslide risk. Firstly, data from past mine slopes were used to construct a knowledge graph. In this graph, each slope is represented as an entity (node), while the similarity relationships between slopes are defined as relations (edges) (e.g., similar and dissimilar).

Then, a translation-based embedding model was employed to represent the nodes and relations of the knowledge graph in a vector space. This allows the similarity between a new slope and past slopes to be calculated. The model evaluates how closely the new slope resembles slopes that have experienced landslides and how different it is from stable slopes, producing a risk value accordingly.

Finally, the landslide risk is calculated using the Landslide Matching Rate (LMR) metric.

$LMR = (L_s + S_d) / (2 \times NS)$

In this formula, the similarity of the new slope to past landslide slopes and its difference from stable slopes are considered to obtain a risk ratio, which is then used to predict the landslide risk for the slope.

2. Gradient Boosting Decision Tree

Gradient Boosting Decision Tree (GBDT) is a machine learning method that combines multiple decision tree models sequentially to improve prediction accuracy. In this approach, each new tree learns from the errors of previous models, aiming to reduce these errors. This step-by-step refinement results in more accurate predictions.

In this study, the GBDT algorithm was used to classify and predict landslide risk based on the extracted features.

3. Crossed Features

Crossed Features, or “feature interactions,” involve combining two or more features to create a new feature. The purpose is to capture stronger and more meaningful relationships that individual features alone may not fully represent.

Some features that may be considered for predicting landslide risk include:

Slope angle (Slope Angle)

Soil type (Soil Type)

Rainfall amount (Rainfall)

Individually, Slope Angle or Rainfall may not provide complete information about risk. However, when evaluated together:

High slope + heavy rainfall → higher risk

Low slope + heavy rainfall → lower risk

This is where crossed features come into play. The model uses these combined features to provide more accurate landslide risk predictions.

4. Use of Semantic Network via Knowledge Graph

In the study, the Landslide Semantic Network was constructed using a knowledge graph, which functions as follows in landslide risk prediction:

Nodes (Entities):

Data from past slopes are represented as individual nodes.

Example:

Slope_A, Slope_B, Slope_C

Edges (Relations):

Relationships between slopes (similarity or dissimilarity) are represented as edges.

Example:

Slope_A → Slope_B (similar)

Slope_A → Slope_C (dissimilar)

Translation-based Embedding:

Nodes and relations in the knowledge graph are represented in a vector space.

This allows the model to compute similarities and differences between a new slope and past slopes.

Risk Prediction:

When a new slope is evaluated, the semantic network compares it with past slopes.

The slope's similarity to landslide-affected slopes and its difference from stable slopes are calculated, and a risk value is generated using the Landslide Matching Rate (LMR).

Summary:

The knowledge graph is used to construct the semantic network and visualize relationships.

The semantic network prevents the model from relying solely on individual features and allows it to account for relationships between slopes, resulting in more accurate landslide risk predictions.

Code :

Project_ai.ipynb

Dosya Düzenle Göster Ekle Çalışma zamanı Araçlar Yardım

Komutlar + Kod + Metin Tümünü çalıştır

[2] 0 sn.

```python
# Importlar
import networkx as nx                 # Knowledge Graph ve semantic network oluşturmak için graph kütüphanesi
import numpy as np                   # Sayısal hesaplamalar ve vektör işlemleri için kullanılan kütüphane
import pandas as pd                  # Veri tabloları (DataFrame) oluşturmak ve veri işlemek için kullanılan kütüphane
import matplotlib.pyplot as plt      # Grafik ve görselleştirme çizmek için kullanılan kütüphane
from sklearn.ensemble import GradientBoostingClassifier  # Gradient Boosting Decision Tree makine öğrenmesi modeli

# ----------------------------
# 1. Knowledge Graph Oluşturma
# ----------------------------

G = nx.DiGraph()  # Yönlü bir grafik oluşturuyoruz (Directed Graph). Yani ilişkilerin yönü var (A → B)

slopes = ["Slope_A","Slope_B","Slope_C","Slope_D"]
# Knowledge graph içindeki düğümler (nodes). Burada her düğüm bir maden yamacını temsil ediyor

for s in slopes:        # slopes listesindeki her yamaç için döngü başlatıyoruz
    G.add_node(s)       # grafiğe bir düğüm (node) ekliyoruz

edges = [
    ("Slope_A","Slope_B","similar"),       # Slope_A ile Slope_B benzer özellikte
    ("Slope_A","Slope_C","dissimilar"),    # Slope_A ile Slope_C farklı özellikte
    ("Slope_B","Slope_D","similar"),       # Slope_B ile Slope_D benzer
    ("Slope_C","Slope_D","dissimilar")     # Slope_C ile Slope_D farklı
]

for u,v,r in edges:                 # edges listesindeki her ilişkiyi döngüyle alıyoruz
    G.add_edge(u,v,relation=r)     # grafiğe kenar (edge) ekliyoruz ve ilişki türünü attribute olarak kaydediyoruz
```

Project_ai.ipynb

Dosya Düzenle Göster Ekle Çalışma zamanı Araçlar Yardım

Komutlar + Kod + Metin Tümünü çalıştır

[2] 0 sn.

```python
# 2. Landslide Feature Data
# -----------------------------

data = pd.DataFrame({               # Heyelan risk tahmini için örnek veri seti oluşturuyoruz

"slope":[                           # Yamaç isimleri
"Slope_A",
"Slope_B",
"Slope_C",
"Slope_D"
],

"slope_angle":[35,20,40,18],       # Yamaç eğim açıları (derece cinsinden)

"rainfall":[120,80,140,70],        # Yağış miktarı (örnek değerler)

"soil_loose":[1,0,1,0],            # Toprak gevşekliği (1 = gevşek, 0 = sağlam)

"landslide":[1,0,1,0]              # Gerçek heyelan durumu (1 = heyelan var, 0 = yok)

})

# -----------------------------
# 3. Cross Feature Oluşturma
# -----------------------------

data["rainfall_slope_cross"] = data["rainfall"] * data["slope_angle"]
# Cross feature oluşturuyoruz
# Yağış ve eğim açısını çarparak yeni bir özellik oluşturuyoruz
# Bu özellik iki faktörün birlikte etkisini temsil eder
```

Project_ai.ipynb

Dosya Düzenle Göster Ekle Çalışma zamanı Araçlar Yardım

Komutlar + Kod + Metin Tümünü çalıştır

[2] 0 sn.

```python
# ------------------------------
# 3. Cross Feature Oluşturma
# ------------------------------

data["rainfall_slope_cross"] = data["rainfall"] * data["slope_angle"]
# Cross feature oluşturuyoruz
# Yağış ve eğim açısını çarparak yeni bir özellik oluşturuyoruz
# Bu özellik iki faktörün birlikte etkisini temsil eder

# ------------------------------
# 4. Gradient Boosting Model
# ------------------------------

X = data[["slope_angle","rainfall","soil_loose","rainfall_slope_cross"]]
# Makine öğrenmesi modeline verilecek giriş özellikleri (features)

y = data["landslide"]
# Modelin tahmin etmeye çalışacağı hedef değişken (label)

model = GradientBoostingClassifier()
# Gradient Boosting Decision Tree modelini oluşturuyoruz

model.fit(X,y)
# Modeli veri ile eğitiyoruz (training)

data["predicted_risk"] = model.predict(X)
# Eğitilen model ile her yamaç için heyelan risk tahmini yapıyoruz
# Sonuçları predicted_risk sütununa ekliyoruz

# ------------------------------
```

Project_ai.ipynb

Dosya Düzenle Göster Ekle Çalışma zamanı Araçlar Yardım

Komutlar + Kod + Metin Tümünü çalıştır

[2] 0 sn.

```python
# 5. Sonuç Tablosu
# ------------------------------

result_table = data[[
"slope",                        # yamaç adı
"slope_angle",                  # eğim açısı
"rainfall",                     # yağış miktarı
"soil_loose",                   # toprak durumu
"rainfall_slope_cross",         # cross feature
"predicted_risk"                # modelin tahmin ettiği heyelan riski
]]

print(result_table)
# oluşturulan sonuç tablosunu ekrana yazdırıyoruz

# ------------------------------
# 6. Landslide Semantic Network
# ------------------------------

pos = nx.spring_layout(G,seed=42)
# düğümlerin grafikteki konumlarını hesaplayan algoritma
# spring layout fiziksel yay modeli kullanır

colors = ["green" if G[u][v]["relation"]=="similar" else "red" for u,v in G.edges()]
# kenar renklerini belirliyoruz
# similar ilişkiler yeşil
# dissimilar ilişkiler kırmızı

labels = nx.get_edge_attributes(G,"relation")
# kenarların üzerindeki relation etiketlerini alıyoruz
```

Project_ai.ipynb

Dosya Düzenle Göster Ekle Çalışma zamanı Araçlar Yardım

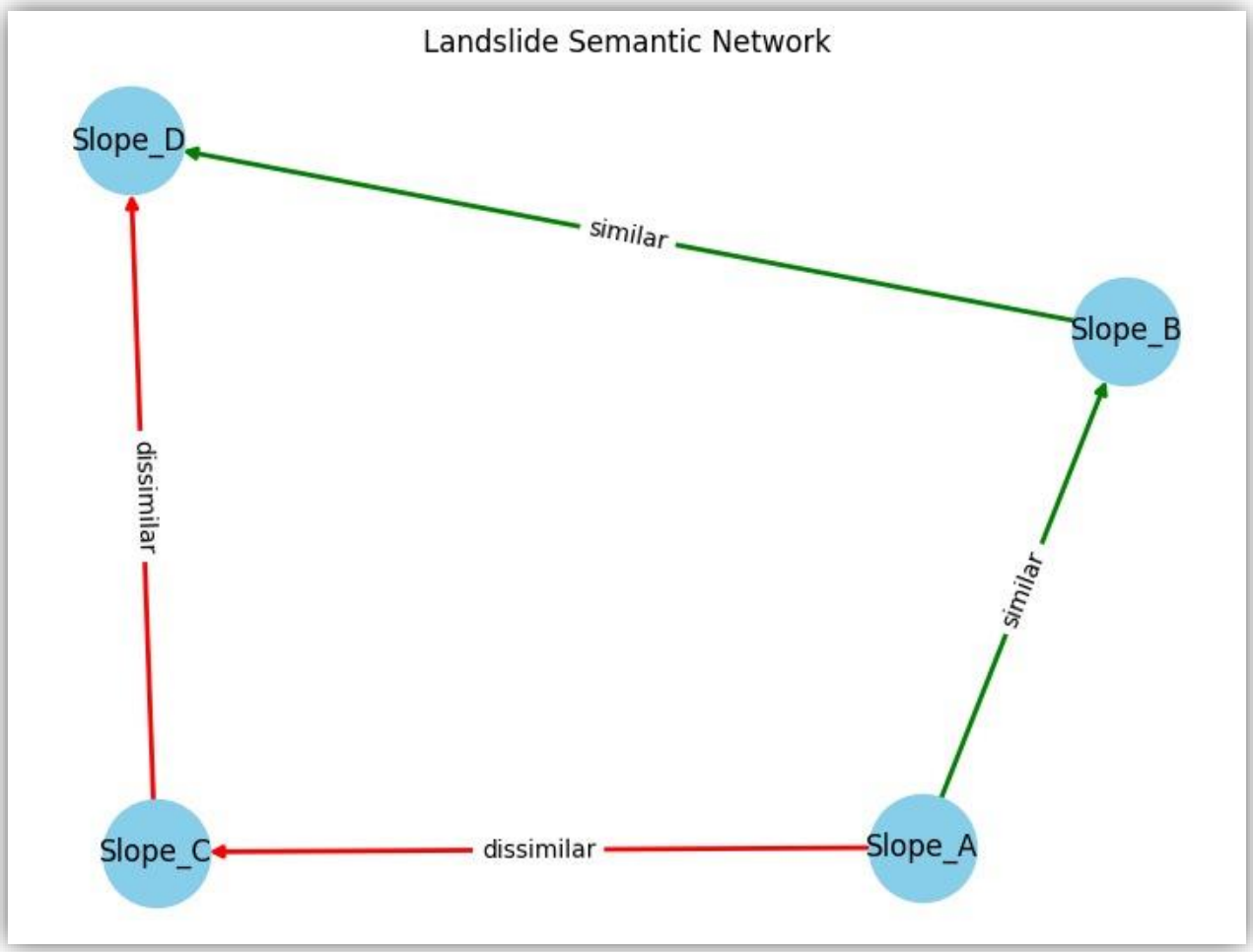
Komutlar + Kod + Metin Tümünü çalıştır

[2] 0 sn.

```python
colors = ["green" if G[u][v]["relation"]=="similar" else "red" for u,v in G.edges()]
# kenar renklerini belirliyoruz
# similar ilişkiler yeşil
# dissimilar ilişkiler kırmızı

labels = nx.get_edge_attributes(G,"relation")
# kenarların üzerindeki relation etiketlerini alıyoruz

plt.figure(figsize=(7,5))
# çizilecek grafiğin boyutunu ayarlıyoruz

nx.draw(
G,
pos,
with_labels=True,          # düğüm isimlerini göster
node_color="skyblue",      # düğüm rengi
node_size=2000,            # düğüm büyüklüğü
edge_color=colors,         # kenar renkleri
width=2                    # kenar kalınlığı
)

nx.draw_networkx_edge_labels(G,pos,edge_labels=labels)
# kenarların üzerine relation etiketlerini yazdırıyoruz

plt.title("Landslide Semantic Network")
# grafiğin başlığını ekliyoruz

plt.show()
# grafiği ekranda gösteriyoruz
```

Result:

Landslide Risk Prediction Results

	slope	slope_angle	rainfall	soil_loose	rainfall_slope_cross	predicted_risk
0	Slope_A	35	120	1	4200	1
1	Slope_B	20	80	0	1600	0
2	Slope_C	40	140	1	5600	1
3	Slope_D	18	70	0	1260	0

Graph :